

drzewo BST-splay

```
global BST,qu
BST = [[None,A[0],None, None]]
qu = 0    #indeks korzenia

def searchST(key):
    n = qu
    while BST[n][1] != key :
        if key > BST[n][1]:
            if BST[n][3]: n = BST[n][3]
            else: return n,3
        else :
            if BST[n][2]: n = BST[n][2]
            else: return n,2
    return n,1    #1==jest; 2==nowy lewo;
                  3==nowy prawo

def inST(key):
    n,j = searchST(key)
    if j != 1:
        BST.append([n,key,None,None])
        BST[n][j] = len(BST) -1
    else:
        print(f"Klucz {key} już jest!")

#Budujemy drzewo BST
for i in range(1,len(A)): inST(A[i])
```

```
#Funkcja rotacji - syn w miejsce ojca
def rotST(n):
    global qu
    y = BST[n][0]; z = BST[y][0]
    if z != None:
        if y==BST[z][2]: BST[z][2]= n
        else: BST[z][3]= n
    BST[n][0] = z
    if n==BST[y][2]: q = 2
    else: q = 3
    son = BST[n][5-q]
    BST[y][0]= n; BST[n][5-q]= y
    if son != None:
        BST[y][q] = son
        BST[son][0] = y
    else: BST[y][q] = None
    if z==None: qu = n
    return z

#Funkcja wyciąga 'key' do korzenia
def splayST(key):
    n,j = searchST(key)
    if BST[n][0]==None :
        return n
    z=0
    while z != None :
        z = rotST(n)
    obraz()
    return n
```

algorytm delete(key) na BST

```
def minST(q):
    while BST[q][2] != None:
        q = BST[q][2]
    return q
```

#Usuwanie węzła o kluczu 'key'

```
def delST(key):
    global qu
    n, j = searchST(key)
    if j != 1: return
    if BST[n][2] == None or
    BST[n][3] == None:
        y = n
    else: y = minST(BST[n][3])
    if BST[y][2] != None:
        x = BST[y][2]
    else: x = BST[y][3]
    if x != None :
        BST[x][0] = BST[y][0]
    z = BST[y][0]
    if z == None : qu = y
    else:
        if y == BST[z][2]:
            BST[z][2] = x
        else: BST[z][3] = x
```

```
def listST():
    for i in range(len(BST)):
        print('%5i -' %i, BST[i])
```

```
if y == n: BST[n] = [None]
else:
    BST[n][1] = BST[y][1]
    BST[y], BST[n] = BST[n], [None]
    z = BST[y][0]
    if z == None : qu = y
    else:
        if BST[z][2] == n:
            BST[z][2] = y
        else: BST[z][3] = y
    for kid in (BST[y][2], BST[y][3]):
        if kid != None:
            BST[kid][0] = y
return
```

```

def init(qu):
    global L
    L={}
    for i in range(25):
        for j in range(45):
            L[i,j]=' '
    L[1,22]=str(BST[qu][1])
    L[2,22]='0'

def rysuj():
    for i in range(25):
        L2=' '
        for j in range(45):
            L2+=L[i,j]
        print(L2)

def draw(x,y,lr,n,deep):
    global L
    d='/'
    if deep>4: deep=4
    if lr==1:d='\\'
    for i in range(1,6-deep):
        L[x+i,y+lr*i]=d
    x+=6-deep;y+=lr*(6-deep)
    L[x,y]='0'
    L[x-1,y]=str(BST[n][1])
    return(x,y)

```

rysowanie drzewa BST

```

def Down(x,y,lr,n,deep):
    global dMax,ws
    if deep>0:
        x,y=draw(x,y,lr,n,deep)
    if BST[n][2:]==[None,None] :
        if deep>dMax:
            dMax=deep; ws=n
        return
    if BST[n][2] != None:
        Down(x,y,-1,BST[n][2],deep+1)
    if BST[n][3] != None:
        Down(x,y,1,BST[n][3],deep+1)
    return

def obraz():
    global dMax,ws
    dMax=0; ws=0
    init(qu)
    Down(2,22,1,qu,0)
    rysuj()

obraz()

```